PBL ASSIGNMENT REPORT

Assignment 4

SDLC Foundations: High-Level Design and Low-Level Design

Movie Ticket Booking System

Name: Manaswini Kusampudi

Roll Number: 24WU0101061

Semester: 5th Semester

Class: CSE-Rhinos

Department of Computer Science and Engineering

Academic Year 2026

TABLE OF CONTENTS

1. Abstract

2. Assignment Definition

3. Problem Statement and Scope

4. Objectives

5. Requirements Analysis

6. Assumptions and Constraints

7. High-Level Design

8. Architecture and Module Design

9. Data Flow Design

10. Low-Level Design

11. UML Class Design

12. Booking Sequence Design

13. Database Design

14. API and Interface Contracts

15. Security, Reliability, and Scalability

16. Traceability and Review Checklist

17. Conclusion

18. References

1. Abstract

This document presents the High-Level Design (HLD) and Low-Level Design (LLD) for a Movie Ticket Booking System. The design translates business requirements into a buildable software structure before implementation begins. The HLD defines the system boundary, major services, modules, data flow, external integrations, storage choices, and deployment concerns. The LLD refines those decisions into domain classes, booking interactions, database entities, API contracts, business rules, validation rules, and error handling. The system supports movie discovery, theatre and showtime browsing, seat availability, temporary seat locking, payment, booking confirmation, ticket delivery, cancellation, and administration. The diagrams are provided as rendered UML/architecture images and editable Mermaid source files.

2. Assignment Definition

Item	Description
Assignment number	4
Part	SDLC Foundations
Topic	High-level and low-level design (HLD/LLD)
Task	Turn requirements into a buildable design
System	Movie Ticket Booking System
HLD deliverables	Architecture, modules, data flow, external integrations
LLD deliverables	Class diagram, sequence diagram, database schema, API contracts
Tools	Draw.io / PlantUML equivalent; editable Mermaid sources included
Industry lesson	Design documents are reviewed and signed off before coding starts

3. Problem Statement and Scope

Customers need a reliable way to discover movies, choose a theatre and show, select seats, pay securely, and receive a ticket. The central consistency challenge is preventing two customers from booking the same seat during concurrent checkout. Theatre administrators need to maintain movie metadata, screens, seats, show schedules, pricing, and operational reports. The design must therefore separate browsing from inventory mutation, use temporary seat locks, make payment and booking states explicit, and preserve an auditable record of every confirmed transaction.

3.1 In scope

- Customer registration and authentication
- Movie, theatre, screen, seat, and showtime discovery
- Availability lookup
- Temporary seat locks with expiry
- Checkout and payment gateway integration
- Booking confirmation and e-ticket notification
- Booking history and cancellation/refund request
- Administrative management of movies, theatres, screens, shows, and pricing
- Operational logging, metrics, and audit events

3.2 Out of scope

- Cinema staff point-of-sale hardware
- Physical ticket scanning implementation
- Recommendation engine
- Multi-currency settlement and tax engine
- Full payment-card storage; card data remains with the payment gateway

4. Objectives

- Convert functional and quality requirements into an implementation-ready architecture.
- Define clear module ownership and service boundaries.
- Model the complete booking flow from search through notification.
- Design a concurrency-safe seat inventory strategy.

• Provide a normalized relational schema for durable booking data.

• Define interfaces that can be implemented and tested independently.

• Document security, reliability, scalability, and sign-off criteria before coding.

5. Requirements Analysis

5.1 Functional requirements

ID	Requirement	Priority
FR-01	Users shall search movies by title, language, genre, and date.	High
FR-02	Users shall browse theatres, screens, shows, prices, and available seats.	High
FR-03	Users shall select one or more seats and obtain a temporary lock.	Critical
FR-04	A lock shall expire automatically after a configured TTL.	Critical
FR-05	Users shall pay through an external payment gateway.	Critical
FR-06	A successful payment shall create one confirmed booking and reserve seats.	Critical
FR-07	The system shall issue a unique booking code and e-ticket.	High
FR-08	Users shall view booking history and request cancellation subject to policy.	Medium
FR-09	Administrators shall manage catalog, theatres, screens, shows, and prices.	High
FR-10	The platform shall record audit events and operational errors.	High

5.2 Non-functional requirements

ID	Quality attribute	Design target
NFR-01	Availability	99.9% monthly availability for booking APIs
NFR-02	Consistency	No double booking for the same show and seat
NFR-03	Performance	Availability response p95 < 300 ms

		under normal load
NFR-04	Security	TLS, hashed credentials, least privilege, token expiry
NFR-05	Scalability	Stateless API instances; horizontal scaling for read traffic
NFR-06	Auditability	Trace booking, payment, lock, cancellation, and admin changes
NFR-07	Recoverability	Durable booking/payment records and database backups
NFR-08	Maintainability	Service boundaries, validation, tests, and documented APIs

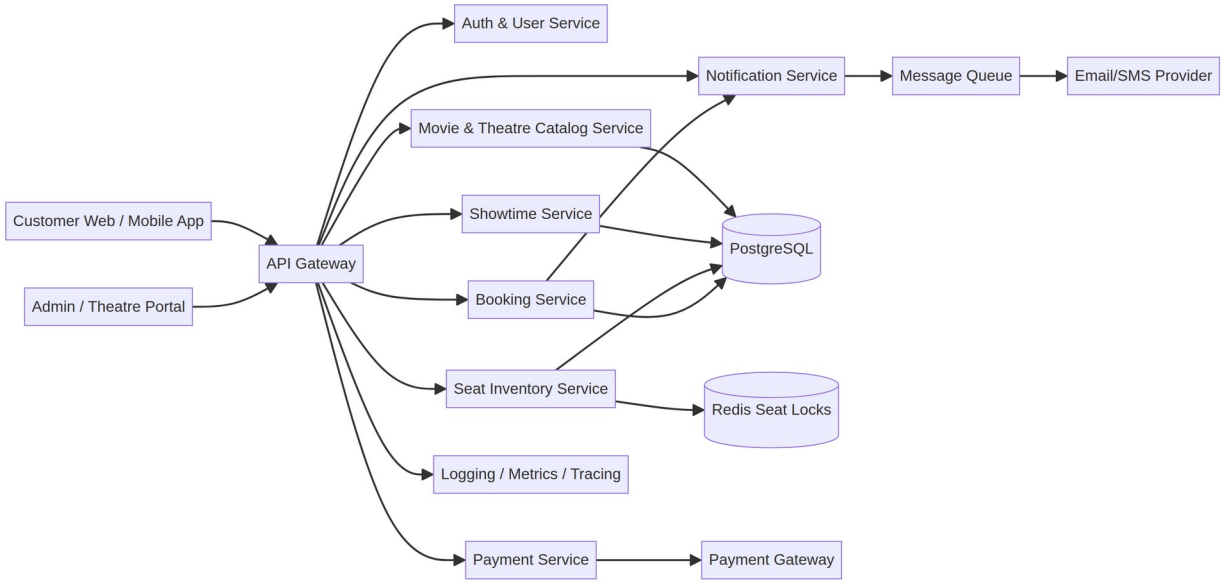
6. Assumptions and Constraints

- A user has a unique email address and may have one active checkout per show.
- The payment gateway returns an idempotency-safe transaction reference.
- Redis is available for short-lived seat locks; PostgreSQL is the source of truth for durable bookings.
- The initial release supports one regional timezone per theatre.
- Seat locks expire after 10 minutes unless converted into a booking.
- A booking is confirmed only after verified payment success.
- The system uses REST/JSON interfaces between clients and services in the first release.
- The diagrams are logical designs; exact cloud products can be selected during implementation planning.

7. High-Level Design

The HLD uses a layered service-oriented architecture. Customer and administrator clients enter through an API Gateway. Stateless domain services handle identity, catalog, shows, inventory, booking, payment coordination, and notification. PostgreSQL stores durable transactional records; Redis manages short-lived seat locks; a message queue decouples notification delivery from the booking request. Observability is cross-cutting and receives structured logs, metrics, traces, and audit events.

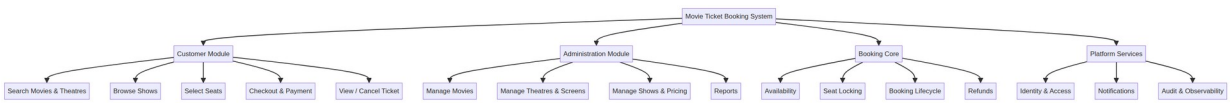
Figure 1. HLD system architecture



The architecture intentionally separates inventory mutation from catalog reads. This reduces the chance that a slow search or catalogue query can interfere with seat locking. Payment remains an external boundary, and the Booking Service reconciles payment results before committing the final reservation.

8. Architecture and Module Design

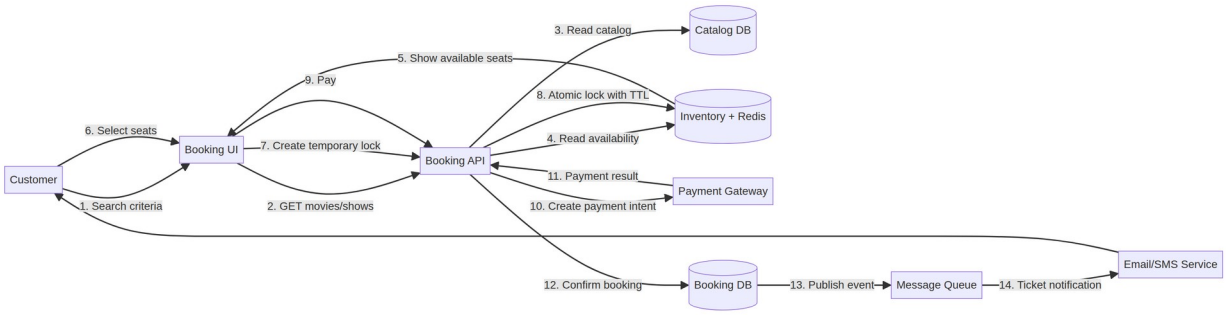
Figure 2. Module decomposition



Module	Responsibilities	Owns
Customer Module	Search, show browsing, seat selection, checkout, ticket history	Customer-facing workflows
Administration Module	Movie, theatre, screen, show, pricing, and reports	Operational configuration
Availability Module	Seat map, availability, lock validation, expiry	Read model and lock coordination
Booking Core	Booking state machine, confirmation, cancellation, refunds	Booking lifecycle
Payment Integration	Payment intent, callback verification, idempotency	Gateway adapter
Notification Module	E-ticket, email/SMS templates, retries	Delivery workflow
Identity and Access	Registration, login, roles, tokens, authorization	Users and permissions
Observability	Logs, metrics, tracing, audit records	Operational evidence

9. Data Flow Design

Figure 3. End-to-end booking data flow



The main flow reads catalog and availability before modifying inventory. A customer selection is converted into a temporary lock with a TTL. Payment is requested only after a successful lock. The Booking Service then verifies the gateway response, writes the durable booking and booking-seat rows, converts the locks into booked state, and publishes a ticket event. Notification delivery is asynchronous so a slow email provider does not keep the customer's booking transaction open.

10. Low-Level Design

The LLD defines the domain objects, state transitions, data ownership, and interfaces required to implement the HLD. The most important design decision is that seat availability is show-

specific: a physical seat can be available for one show and booked for another. Therefore, the booking and lock records always carry both show identity and seat identity.

10.1 Booking state model

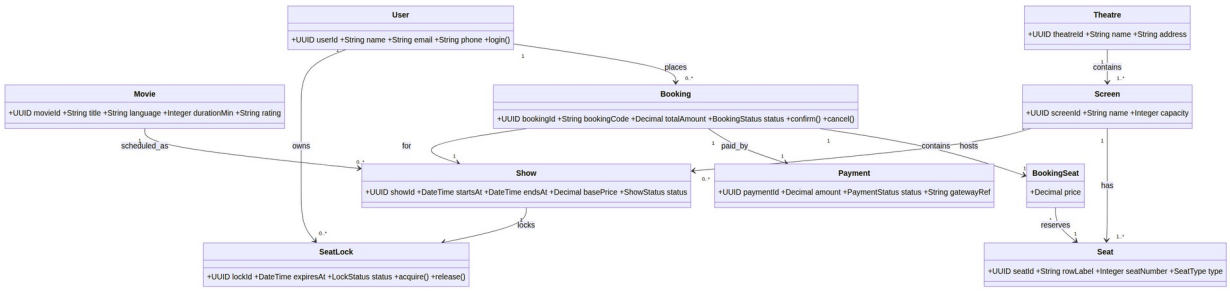
State	Meaning	Allowed transitions
PENDING_PAYMENT	Seats locked; payment not yet confirmed	CONFIRMED, PAYMENT_FAILED, EXPIRED
CONFIRMED	Payment verified and seats permanently reserved	CANCELLED
PAYMENT_FAILED	Gateway or verification failure	PENDING_PAYMENT retry or EXPIRED
EXPIRED	Lock TTL ended before confirmation	None
CANCELLED	Booking cancelled and refund policy applied	None

10.2 Seat-lock rules

- Lock operation must be atomic for all requested seats.
- If any requested seat is unavailable, no partial lock is retained.
- Each lock contains show_id, seat_id, user_id, expiry, and status.
- A background expiry worker marks expired locks and releases Redis keys.
- Confirmation checks that the lock belongs to the requesting user and has not expired.
- Database uniqueness and transactional checks protect against duplicate durable reservations.

11. UML Class Design

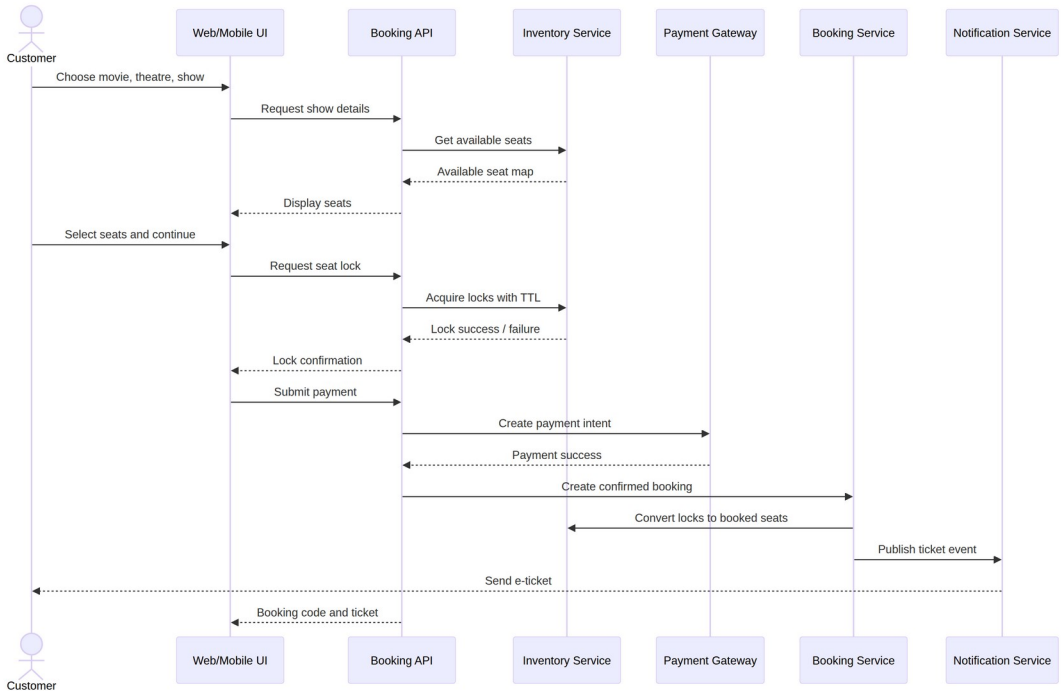
Figure 4. UML domain class diagram



The class model separates catalogue entities (Movie, Theatre, Screen, Seat, Show) from transaction entities (SeatLock, Booking, BookingSeat, Payment). BookingSeat is an associative entity because a booking contains many seats and a seat participates in many bookings across different shows. Payment is one-to-one with a booking in the first-release design, while gateway callbacks are made idempotent by gateway_ref.

12. Booking Sequence Design

Figure 5. UML sequence diagram for seat booking



The sequence shows the success path. Error paths include unavailable seats, expired locks, payment failure, duplicate callback, and notification retry. The API must never report a confirmed booking before the durable booking transaction succeeds.

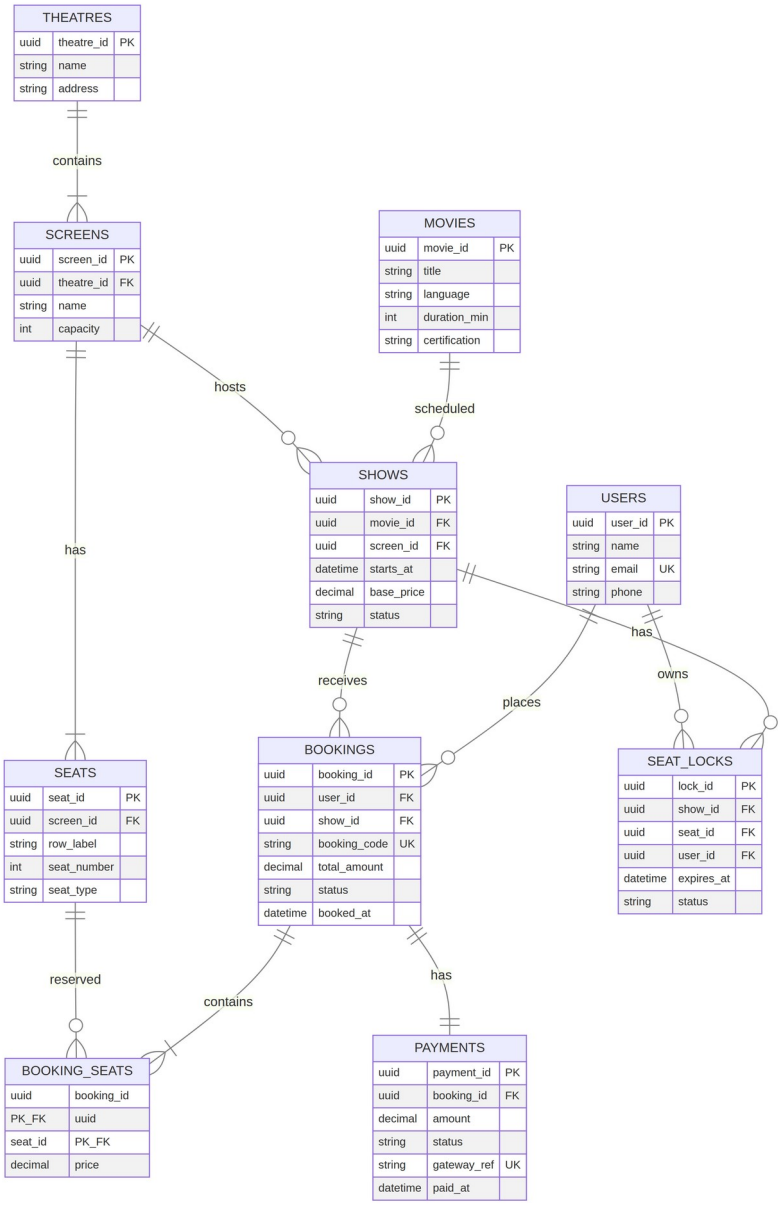
12.1 Error paths

Scenario	System response
Seat lock conflict	Return HTTP 409 with unavailable seat identifiers; do not create partial lock
Lock expiry	Return HTTP 410 or domain error; ask customer to select seats again
Payment failure	Mark payment failed, release locks, return retry-safe response

Payment timeout	Keep booking pending only within policy window; reconcile asynchronously
Duplicate callback	Use gateway reference and idempotency key; return prior outcome
Notification failure	Keep booking confirmed; retry notification through queue

13. Database Design

Figure 6. Relational database schema



PostgreSQL is the durable store for users, catalogue, shows, bookings, payments, and audit-relevant records. Redis is deliberately not shown as the source of truth in the ER model because its keys are temporary coordination state. The schema is normalized around theatre, screen, seat, show, booking, and booking-seat relationships.

Table	Primary key	Important fields / constraints
users	user_id	email UNIQUE; role; password_hash; created_at
movies	movie_id	title; language; duration_min; certification
theatres	theatre_id	name; address
screens	screen_id	theatre_id FK; capacity
seats	seat_id	screen_id FK; row_label; seat_number; seat_type
shows	show_id	movie_id FK; screen_id FK; starts_at; base_price; status
seat_locks	lock_id	show_id + seat_id; user_id; expires_at; status
bookings	booking_id	user_id FK; show_id FK; booking_code UNIQUE; status; total_amount
booking_seats	booking_id + seat_id	Composite key; price snapshot
payments	payment_id	booking_id FK; gateway_ref UNIQUE; status; amount

13.1 Representative SQL DDL

```
CREATE TABLE bookings (
  booking_id UUID PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(user_id),
  show_id UUID NOT NULL REFERENCES shows(show_id),
  booking_code VARCHAR(24) NOT NULL UNIQUE,
  total_amount NUMERIC(10,2) NOT NULL CHECK (total_amount >= 0),
  status VARCHAR(24) NOT NULL,
  booked_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE booking_seats (
  booking_id UUID REFERENCES bookings(booking_id),
  seat_id UUID REFERENCES seats(seat_id),
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
  PRIMARY KEY (booking_id, seat_id)
);
```

14. API and Interface Contracts

Method	Endpoint	Purpose	Success response
POST	/api/v1/auth/login	Authenticate user	200 token payload
GET	/api/v1/movies?date=&city=	Search movies	200 movie list
GET	/api/v1/shows/{showId}/seats	Read seat availability	200 seat map
POST	/api/v1/shows/{showId}/locks	Lock selected seats	201 lock id and expiry
POST	/api/v1/bookings/checkout	Create payment intent	201 checkout id
POST	/api/v1/payments/webhook	Receive verified gateway event	200 acknowledged
GET	/api/v1/bookings/{bookingId}	View booking	200 booking and ticket
POST	/api/v1/bookings/{bookingId}/cancel	Cancel under policy	200 cancellation/refund status
POST	/api/v1/admin/shows	Create or update show	201 show

14.1 Example lock request

```
POST /api/v1/shows/SHOW-123/locks

Idempotency-Key: 3d8d...

Authorization: Bearer <token>


{

  "seatIds": ["SEAT-A-10", "SEAT-A-11"],

  "clientRequestId": "checkout-789"

}
```

14.2 Example response

```
HTTP/1.1 201 Created

{

  "lockId": "LOCK-456",

  "expiresAt": "2026-10-06T02:10:00Z",

  "status": "ACTIVE"

}
```

15. Security, Reliability, and Scalability

15.1 Security

- TLS for all client and service traffic.
- Passwords stored using a slow adaptive hash; never store raw passwords.
- Short-lived access tokens and refresh-token rotation.
- Role-based authorization for customer, theatre-admin, and platform-admin actions.
- Payment-card data delegated to the gateway; store only tokenized references.
- Validate all request fields and enforce ownership checks on bookings and locks.
- Rate-limit login, lock, payment, and webhook endpoints.
- Record security-relevant actions in an append-only audit stream.

15.2 Reliability and scalability

- Use database transactions for booking confirmation.
- Use idempotency keys for checkout and gateway callbacks.
- Use Redis TTLs and an expiry worker for abandoned locks.
- Keep APIs stateless so instances can scale horizontally.
- Cache movie and theatre catalogue reads with explicit invalidation.
- Use queue retries and dead-letter handling for notification failures.
- Monitor lock conflicts, payment failures, booking latency, queue depth, and database health.
- Back up PostgreSQL and test restore procedures.

16. Traceability and Review Checklist

Requirement / decision	HLD/LLD location	Verification method
FR-03 / FR-04 seat locking	Sections 7, 10.2, 12	Concurrency and expiry tests

FR-05 / FR-06 payment and confirmation	Sections 7, 12, 14	Gateway sandbox and idempotency tests
FR-07 e-ticket	Sections 7, 8, 12	Notification event and retry test
NFR-02 no double booking	Sections 10.2, 13	Parallel booking test
NFR-04 security	Section 15.1	Threat model and security review
NFR-05 scalability	Section 15.2	Load test and horizontal scale test
Database integrity	Section 13	Migration and constraint tests
API readiness	Section 14	Contract tests and OpenAPI review

Design review exit criteria: all high-priority requirements have a mapped component and test approach; every diagram has an owner; unresolved assumptions are documented; security and concurrency risks have mitigation owners; and the sign-off table below is completed before coding begins.

17. Conclusion

The Movie Ticket Booking System has been translated from a business problem into a buildable design. The HLD establishes a clear architecture with customer and admin clients, an API Gateway, domain services, durable storage, temporary inventory locks, payment integration, asynchronous notifications, and observability. The LLD turns that architecture into classes, a booking sequence, normalized tables, state transitions, API contracts, validation rules, and error handling.

The design’s most important correctness guarantee is the seat-lock and confirmation flow. A seat is temporarily reserved before payment, the lock expires safely when checkout is abandoned, and only verified payment can convert the lock into a durable booking. The documents should be reviewed and signed off before implementation so that the team can code against an agreed structure rather than discovering architectural decisions late in development.

18. References

- Object Management Group. UML specification and diagram concepts. https://www.omg.org/spec/UML/
- PlantUML documentation. https://plantuml.com/
- Mermaid documentation. https://mermaid.js.org/

• OWASP Application Security Verification Standard. https://owasp.org/www-project-application-security-verification-standard/

• PostgreSQL documentation. https://www.postgresql.org/docs/

• Redis documentation. https://redis.io/docs/